

Editorial: Problem C - Koyomi's Revisions

Setter: Hudson

Problem Overview

Given a string S , answer Q queries for the number of distinct substrings in $S[L \dots R]$.

Difficulty Analysis

Rating: **2700**. This is a well-known hard problem. Naive solution (generating substrings and hashing) is $O(Q \cdot N^2)$, totally impossible. Suffix Automaton construction is $O(N)$, but querying ranges is hard because the structure of the SAM corresponds to the *whole* string.

Solution Approach: Offline Queries + SAM + Segment Tree

We process queries **offline**, sorted by their right endpoint R . As we iterate R from 1 to N , we want to maintain a data structure that allows us to sum up the contribution of valid substrings ending at or before R that started at or after L .

A substring is defined by (l, r) . It contributes to the answer for query $[L, R]$ if $L \leq l \leq r \leq R$. To avoid overcounting duplicates (e.g., "a" appearing at index 1 and 3), we only count the **last occurrence** of each distinct substring. If a distinct string T appears multiple times ending at $r_1, r_2, \dots, r_k$, we only want to count the instance ending at r_k (the rightmost one seen so far).

Algorithm: 1. Build a Suffix Automaton (SAM) for S . 2. We map each node u in the SAM to the distinct substrings represented by that state. These substrings have lengths in range $(len(link(u)), len(u)]$. 3. We need to track the **last end position** of each SAM state. Let $pos(u)$ be the latest end-position of the occurrences of substrings in state u . 4. As we append character $S[R]$: - We traverse up the SAM link tree from the current state corresponding to the prefix $S[1 \dots R]$. - For each state u on this path, the substrings represented by u now have a new end position R . - Previously, they ended at $pos(u)$. We need to "remove" their contribution from $pos(u)$ and "add" it to R . - Specifically, state u represents a range of lengths. The longest string in u has length $len(u)$. The shortest has length $len(link(u)) + 1$. - This effectively updates a range of start positions. 5. This looks like "Color the path to root" in LCT. - We use a **Link-Cut Tree (LCT)** to maintain the SAM link tree. - The "Access" operation in LCT makes the path from the current node to the root a single "heavy" chain (or preferred path). - This corresponds to updating the 'last_pos' for all nodes on the path to R . 6. When the 'last_pos' of a chain changes from old_pos to R : - The substrings covered by this chain have lengths $[min_len, max_len]$. - Their starting positions were $[old_pos - max_len + 1, old_pos - min_len + 1]$. - We subtract 1 from these starting indices in a Segment Tree / Fenwick Tree. - We add 1 to the new starting indices $[R - max_len + 1, R - min_len + 1]$. 7. Answer for query $[L, R]$ is sum in Fenwick Tree range $[L, R]$.

Complexity: $O(N \log^2 N)$ using LCT + SegTree.